

Subject 8: Agent Frameworks & Tool Use

1. The 80/20 Core (Hiring-Critical)

What Actually Matters (20% → 80% Impact)

Included (Production & Interview Critical):

1. **ReAct Pattern Implementation** (35% of value)
 - Reasoning steps before action
 - Observation integration
 - Multi-turn planning and execution
 - Error recovery and replanning
 - Why: ReAct is the foundational pattern; 80% of production agents use this architecture
2. **Tool Design & Schema Definition** (25% of value)
 - Function signatures and descriptions
 - Parameter validation and type safety
 - Error handling and retries
 - Tool composition and dependencies
 - Why: Bad tool design → agent confusion; clear schemas = 3x success rate
3. **Agent Execution Control** (20% of value)
 - Max iterations and timeout handling
 - Budget constraints (token/cost limits)
 - Early stopping conditions
 - Infinite loop detection
 - Why: Uncontrolled agents waste tokens and money; proper control = 90% cost reduction in failure cases
4. **Multi-Tool Orchestration** (10% of value)
 - Tool selection strategies
 - Sequential vs parallel execution
 - State management between tools
 - Dependency resolution
 - Why: Real agents need multiple tools; orchestration is where complexity lives
5. **Agent-Specific Evaluation** (10% of value)
 - Task success rate (not just quality)
 - Tool usage correctness

- Efficiency metrics (steps to solution)
- Cost per successful task
- Why: Agent eval is fundamentally different from single-turn LLM eval

Explicitly Excluded (Low ROI / Interview Anti-Signals):

- ❌ Building agents from scratch (use LangChain, LlamaIndex, etc.)
Why excluded: Reinventing the wheel; frameworks solve 90% of problems
- ❌ Complex planning algorithms (MCTS, A*)
Why excluded: Overkill for most use cases; simple ReAct works for 95%
- ❌ Multi-agent systems (agent teams, debates)
Why excluded: Premature complexity; single agent solves most problems
- ❌ Fine-tuning for agent behavior
Why excluded: Prompt engineering + tool design sufficient for 90% of cases
- ❌ Autonomous agents (no human in loop)
Why excluded: High-risk, not production-ready for most applications

Hiring Signal Justification

What Tier 1 interviewers look for:

- Can you implement a ReAct agent from scratch?
- Do you understand tool schema design and why it matters?
- Have you debugged agent infinite loops or tool misuse?
- Can you explain when NOT to use an agent?
- Do you know how to evaluate agent performance?

What signals "student not engineer":

- "I just use LangChain defaults" (no understanding of internals)
 - No error handling or timeout logic
 - Can't explain the difference between agent and chain
 - Treating agents as magic (no systematic debugging)
 - No understanding of cost implications
-

2. Genius-Level Understanding

Core Mental Models

Perspective 1: Infrastructure Engineer

Agents are stateful, multi-step programs with non-deterministic control flow.

Traditional program:

python

```
# Deterministic control flow
def process_request(input):
    data = fetch_data(input)      # Step 1: Always runs
    result = transform(data)      # Step 2: Always runs
    output = format(result)       # Step 3: Always runs
    return output

# Predictable execution path, fixed steps
```

Agent program:

python

```
# Non-deterministic control flow
def agent_process(input):
    state = initial_state(input)

    while not task_complete(state):
        # LLM decides next action
        action = llm.decide_action(state) # Could be ANY action!

        if action.type == "search":
            observation = search_tool(action.params)
        elif action.type == "calculate":
            observation = calculator_tool(action.params)
        elif action.type == "final_answer":
            return observation
        else:
            # LLM might hallucinate invalid action!
            observation = "Error: Unknown action"
```

```

        state = update_state(state, action, observation)

    return state.answer

# Unpredictable execution path, variable steps
# Could take 1 step or 20 steps
# Could succeed or loop infinitely

```

Key insight: Agents are fundamentally harder to reason about than traditional programs because:

- **Non-deterministic:** Same input can take different execution paths
- **Unbounded:** Could run indefinitely if not controlled
- **Error-prone:** LLM can make invalid tool calls
- **Expensive:** Each step costs tokens

Production implications:

python

```

# Amateur: No control, no error handling
def naive_agent(query):
    state = {"query": query, "history": []}

    while True: # ❌ Infinite loop risk!
        action = llm.decide(state)
        result = execute_tool(action) # ❌ No error handling!
        state["history"].append((action, result))

    if "answer" in result:
        return result["answer"] # ❌ Might never reach here!

# Engineer: Controlled execution with error handling
def robust_agent(query, max_iterations=10, max_cost=0.50):
    state = {"query": query, "history": [], "cost": 0}

    for iteration in range(max_iterations): # ✅ Bounded iterations
        # Cost check
        if state["cost"] > max_cost: # ✅ Budget control
            return {"error": "Budget exceeded", "partial_result": state}

    # Decide next action
    try:
        action = llm.decide(state)

```

```

        state["cost"] += calculate_cost(action)

    # Validate action
    if not is_valid_action(action): # ✅ Validation
        state["history"].append({
            "action": action,
            "observation": "Invalid action, please try again"
        })
        continue

    # Execute tool with timeout
    result = execute_tool_with_timeout(action, timeout=30) # ✅ Timeout

except ToolExecutionError as e: # ✅ Error handling
    result = {"error": str(e)}

state["history"].append({"action": action, "observation": result})

# Check if done
if action.type == "final_answer":
    return result

# Max iterations reached
return {"error": "Max iterations exceeded", "partial_result": state}

```

Perspective 2: Product Engineer

Agents expand the solution space but introduce new failure modes.

Single-turn LLM:

User Query → LLM → Answer

Failure modes:

- Hallucination
- Irrelevant answer
- Refusal

User sees: One response

Agent with tools:

User Query → Agent decides → Tool 1 → Observation

↓

```
Agent reasons → Tool 2 → Observation
↓
Agent reasons → Tool 3 → Observation
↓
Final Answer
```

Failure modes:

- Wrong tool selection
- Invalid tool parameters
- Tool execution errors
- Infinite loops
- Incorrect reasoning
- Hallucination (still!)
- Early stopping (giving up)
- Late stopping (overworking)

User sees: Final answer (success) or error (any step failed)

The Agent Paradox:

- **Upside:** Can solve complex tasks that single-turn LLMs can't
- **Downside:** More steps = more failure points
- **Result:** Agents have LOWER reliability than single-turn (80% vs 95%)

When to use agents:

python

```
# Decision framework
def should_use_agent(task):
    # Use agent if task requires:
    # 1. External data access
    if requires_current_data(task):
        return True # e.g., "What's the weather today?"

    # 2. Multiple steps
    if requires_multiple_steps(task):
        return True # e.g., "Book a flight and hotel for cheapest total"

    # 3. Tool use
    if requires_computation(task):
        return True # e.g., "Calculate compound interest over 20 years"

    # Don't use agent if:
    # 1. Simple question answering
```

```

if is_simple_qa(task):
    return False # Use RAG instead

# 2. Creative generation
if is_creative_task(task):
    return False # Use single-turn LLM

# 3. High reliability requirement
if requires_99_reliability(task):
    return False # Agents are less reliable

return False # Default: don't use agent

# Examples
should_use_agent("What is the capital of France?") # False (use RAG)
should_use_agent("Write a poem about spring") # False (use single-turn)
should_use_agent("Book me the cheapest flight from NYC to LAX next week") # True (needs agent)
should_use_agent("Calculate the ROI of investing $10k at 7% for 30 years") # True (needs agent)

```

User experience design:

python

```

# Show agent progress to user
def agent_with_progress(query):
    state = initial_state(query)

    # Show initial message
    show_user("🤔 Thinking about your request...")

    while not done(state):
        action = llm.decide(state)

        # Show what agent is doing
        if action.type == "search":
            show_user(f"🔍 Searching for: {action.params['query']}")
        elif action.type == "calculate":
            show_user(f"🧮 Calculating: {action.params['expression']}")

        observation = execute_tool(action)
        state = update_state(state, action, observation)

    show_user(f"✅ {state.answer}")

    # Show agent's reasoning (optional)

```

```
if user_wants_explanation:
    show_reasoning_trace(state.history)
```

Perspective 3: Researcher

ReAct (Reasoning + Acting) outperforms pure reasoning or pure acting.

Pure Reasoning (Chain-of-Thought):

Query: "What is the population of the capital of France?"

LLM: Let me think through this step by step:

1. The capital of France is Paris
2. Paris has a population of about 2.2 million
3. Therefore, the answer is 2.2 million

Problem: LLM doesn't have access to current data!

Actual population: 2.16 million (close, but could be outdated)

Pure Acting (No reasoning):

Query: "What is the population of the capital of France?"

LLM: [calls search("population of France")]

Observation: "France has a population of 67 million"

LLM: The answer is 67 million

Problem: LLM didn't understand the query asked for CAPITAL, not country!

ReAct (Reasoning + Acting):

Query: "What is the population of the capital of France?"

Thought: I need to first identify the capital of France, then look up its population.

Action: search("capital of France")

Observation: "Paris is the capital of France"

Thought: Now I know the capital is Paris. Let me search for its population.

Action: search("population of Paris France")

Observation: "The city of Paris has a population of approximately 2.16 million"

Thought: I have the answer. The population of Paris is 2.16 million.

Answer: 2.16 million

✓ Correct! Reasoning guides action selection, actions ground reasoning in reality.

Key insight: Reasoning provides planning, acting provides grounding. Together they solve complex tasks.

Empirical results (from ReAct paper):

- Chain-of-Thought: 45% success rate
- Pure Acting: 38% success rate
- ReAct: 67% success rate

Why ReAct works:

1. **Reasoning before acting:** Prevents impulsive wrong tool calls
2. **Acting provides grounding:** Prevents hallucination
3. **Reasoning after acting:** Integrates observations into plan
4. **Iterative refinement:** Can recover from mistakes

Perspective 4: Hiring Manager

Show me you understand agent failure modes and have debugging strategies.

Red flags:

- "Agents always work" (no understanding of failure modes)
- No timeout or iteration limits
- Can't explain when NOT to use agents
- No tool usage metrics or monitoring
- Treating agents as black boxes

Green flags:

- Systematic debugging of agent loops
- Tool usage analytics (which tools, success rates)
- Clear criteria for agent vs non-agent solutions
- Error handling at every step
- Cost and latency monitoring

Interview question: "Your agent is stuck in a loop calling the same tool repeatedly. How do you debug and fix this?"

Strong answer:

```
1. **Reproduce the issue**
  - Get the exact query that causes the loop
  - Log all tool calls and observations

2. **Analyze the pattern**
  - Which tool is being called repeatedly?
  - What are the parameters?
  - What observations is it getting?

3. **Identify root cause**
  Common causes:
  - Tool returns unhelpful observation
  - LLM doesn't understand how to use observation
  - Goal is impossible but LLM doesn't realize

4. **Fix strategies**
  a) Improve tool output format
     - Make observations more actionable
     - Include explicit "success" or "failure" indicators

  b) Improve agent prompting
     - Add examples of tool usage
     - Add explicit stopping conditions
     - Add "if you've tried this tool 3 times, try something else"

  c) Add loop detection
  ```python
 def detect_loop(history, window=3):
 recent = history[-window:]
 if len(recent) < window:
 return False

 # Check if all recent actions are the same
 actions = [h["action"]["type"] for h in recent]
 if len(set(actions)) == 1:
 return True # Loop detected!

 return False
```

d) Add forced intervention

```
if detect_loop(state["history"]):
 state["history"].append({
 "system_message": "You've tried this tool 3 times. Try a different approach."
 })
```

### 1. Prevent recurrence

- Add to test suite (regression test)
- Add monitoring for loop detection in production
- Alert if average iterations > threshold

```
'''
'''
```

---

## Advanced Production Use Cases

### Case 1: Research Agent (Multi-Step Information Gathering)

*Problem:* User asks complex question requiring multiple lookups and synthesis.

*Solution:* ReAct agent with web search and calculator tools.

python

```
agents/research_agent.py
from typing import Dict, List, Any, Optional
from dataclasses import dataclass
import json
from openai import OpenAI

@dataclass
class Tool:
 name: str
 description: str
 parameters: Dict[str, Any]
 function: callable

@dataclass
class Action:
 tool: str
 parameters: Dict[str, Any]
```

```

reasoning: str

@dataclass
class Observation:
 result: Any
 success: bool
 error: Optional[str] = None

class ReActAgent:
 def __init__(self, tools: List[Tool], max_iterations: int = 10):
 self.tools = {tool.name: tool for tool in tools}
 self.max_iterations = max_iterations
 self.client = OpenAI()

 def run(self, query: str) -> Dict:
 """Run ReAct agent"""

 state = {
 "query": query,
 "scratchpad": [], # ReAct thought/action/observation traces
 "iteration": 0,
 "final_answer": None
 }

 while state["iteration"] < self.max_iterations:
 state["iteration"] += 1

 # Generate thought and action
 action = self._generate_action(state)

 if action.tool == "final_answer":
 state["final_answer"] = action.parameters["answer"]
 break

 # Execute action
 observation = self._execute_action(action)

 # Update scratchpad
 state["scratchpad"].append({
 "thought": action.reasoning,
 "action": f"{action.tool}({json.dumps(action.parameters)})",
 "observation": observation.result if observation.success else observation.error
 })

 # Check for loops
 if self._detect_loop(state["scratchpad"]):

```

```

 state["final_answer"] = {
 "error": "Agent stuck in loop",
 "partial_result": state["scratchpad"]
 }
 break

 if state["final_answer"] is None:
 state["final_answer"] = {
 "error": "Max iterations reached",
 "partial_result": state["scratchpad"]
 }

 return {
 "answer": state["final_answer"],
 "iterations": state["iteration"],
 "scratchpad": state["scratchpad"]
 }

def _generate_action(self, state: Dict) -> Action:
 """Use LLM to decide next action"""

 # Build prompt with ReAct format
 prompt = self._build_react_prompt(state)

 # Get LLM response
 response = self.client.chat.completions.create(
 model="gpt-4",
 messages=[
 {"role": "system", "content": self._get_system_prompt()},
 {"role": "user", "content": prompt}
],
 temperature=0
)

 # Parse response
 action_text = response.choices[0].message.content
 return self._parse_action(action_text)

def _build_react_prompt(self, state: Dict) -> str:
 """Build ReAct-style prompt"""

 prompt = f"""Question: {state['query']}

```

You have access to these tools:

```
{self._format_tools()}
```

Use the following format:

Thought: [your reasoning about what to do next]

Action: [tool\_name]

Action Input: {"param1": "value1", "param2": "value2"}}

Or when you have the final answer:

Thought: [your reasoning about the answer]

Action: final\_answer

Action Input: {"answer": "your final answer"}}

Previous steps:

"""

```
Add scratchpad
for i, step in enumerate(state["scratchpad"], 1):
 prompt += f"\nThought {i}: {step['thought']}"
 prompt += f"\nAction {i}: {step['action']}"
 prompt += f"\nObservation {i}: {step['observation']}\n"

prompt += f"\nThought {len(state['scratchpad']) + 1}:"

return prompt

def _format_tools(self) -> str:
 """Format tool descriptions"""
 tool_descriptions = []
 for name, tool in self.tools.items():
 params = json.dumps(tool.parameters, indent=2)
 tool_descriptions.append(
 f"- {name}: {tool.description}\n Parameters: {params}"
)
 return "\n".join(tool_descriptions)

def _parse_action(self, action_text: str) -> Action:
 """Parse LLM output into Action"""

 lines = action_text.strip().split('\n')

 reasoning = ""
 tool = None
 parameters = {}

 for line in lines:
 if line.startswith("Thought:"):
 reasoning += line + "\n"
```

```

 reasoning = line.replace("Thought:", "").strip()
 elif line.startswith("Action:"):
 tool = line.replace("Action:", "").strip()
 elif line.startswith("Action Input:"):
 input_str = line.replace("Action Input:", "").strip()
 try:
 parameters = json.loads(input_str)
 except json.JSONDecodeError:
 parameters = {"raw": input_str}

 if not tool:
 # LLM didn't follow format, force final_answer
 tool = "final_answer"
 parameters = {"answer": action_text}

 return Action(tool=tool, parameters=parameters, reasoning=reasoning)

def _execute_action(self, action: Action) -> Observation:
 """Execute tool and return observation"""

 if action.tool not in self.tools:
 return Observation(
 result=None,
 success=False,
 error=f"Unknown tool: {action.tool}"
)

 tool = self.tools[action.tool]

 try:
 result = tool.function(**action.parameters)
 return Observation(result=result, success=True)
 except Exception as e:
 return Observation(
 result=None,
 success=False,
 error=f"Tool execution failed: {str(e)}"
)

def _detect_loop(self, scratchpad: List[Dict], window: int = 3) -> bool:
 """Detect if agent is stuck in loop"""

 if len(scratchpad) < window:
 return False

 recent_actions = [step["action"] for step in scratchpad[-window:]]

```

```

 # If all recent actions are the same, likely a loop
 if len(set(recent_actions)) == 1:
 return True

 return False

def _get_system_prompt(self) -> str:
 return """You are a helpful research assistant that can use tools to answer ques

Important rules:
1. Always think before acting
2. Use tools to get current information
3. If a tool fails, try a different approach
4. If you've tried the same tool 3 times without progress, try something else
5. When you have a complete answer, use the final_answer tool"""

Define tools
def web_search(query: str) -> str:
 """Search the web (mock implementation)"""
 # In production, use actual search API
 return f"Search results for '{query}': [mock results]"

def calculator(expression: str) -> float:
 """Calculate mathematical expression"""
 try:
 # Safe eval (in production, use a proper math parser)
 return eval(expression, {"__builtins__": {}}, {})
 except Exception as e:
 raise ValueError(f"Invalid expression: {e}")

Create agent
tools = [
 Tool(
 name="web_search",
 description="Search the web for current information",
 parameters={"query": "string"},
 function=web_search
),
 Tool(
 name="calculator",
 description="Calculate mathematical expressions",
 parameters={"expression": "string (e.g., '2 + 2' or '10 * 1.07**30')"},
 function=calculator
)
]

```



```

agent = ReActAgent(tools, max_iterations=10)

Usage
result = agent.run("What is 10000 invested at 7% annual return for 30 years?")

print(f"Answer: {result['answer']}")
print(f"Iterations: {result['iterations']}")
print("\nScratchpad:")
for step in result['scratchpad']:
 print(f"\nThought: {step['thought']}")
 print(f"Action: {step['action']}")
 print(f"Observation: {step['observation']}")

Output:
Thought: I need to calculate compound interest. I'll use the calculator.
Action: calculator({"expression": "10000 * (1.07 ** 30)"})
Observation: 76122.55
#
Thought: I have the answer.
Action: final_answer({"answer": "$76,122.55"})
#
Answer: $76,122.55
Iterations: 2

```

**Impact:** Can solve multi-step problems that require both reasoning and tool use.

## Case 2: Function Calling Agent (Structured Tool Use)

*Problem:* ReAct with string parsing is brittle. LLM might not follow format.

*Solution:* Use OpenAI function calling for structured tool invocation.

python

```

agents/function_calling_agent.py
from openai import OpenAI
from typing import List, Dict, Any, Callable
import json

class FunctionCallingAgent:
 def __init__(self, tools: List[Dict[str, Any]], max_iterations: int = 10):
 self.client = OpenAI()
 self.tools = tools
 self.tool_functions = {} # Map tool names to actual functions

```

```

self.max_iterations = max_iterations

def register_tool(self, name: str, function: Callable):
 """Register a tool's implementation"""
 self.tool_functions[name] = function

def run(self, query: str) -> Dict:
 """Run agent with function calling"""

 messages = [
 {
 "role": "system",
 "content": "You are a helpful assistant that can use tools to answer que
 },
 {
 "role": "user",
 "content": query
 }
]

 iteration = 0

 while iteration < self.max_iterations:
 iteration += 1

 # Call LLM with function calling
 response = self.client.chat.completions.create(
 model="gpt-4",
 messages=messages,
 tools=self.tools,
 tool_choice="auto"
)

 assistant_message = response.choices[0].message

 # Check if LLM wants to call a function
 if assistant_message.tool_calls:
 # LLM decided to use a tool
 messages.append(assistant_message)

 # Execute each tool call
 for tool_call in assistant_message.tool_calls:
 function_name = tool_call.function.name
 function_args = json.loads(tool_call.function.arguments)

 # Execute tool

```



```

 "description": "The temperature unit"
 }
},
"required": ["location"]
}
}
},
{
 "type": "function",
 "function": {
 "name": "search_flights",
 "description": "Search for flights between two cities",
 "parameters": {
 "type": "object",
 "properties": {
 "origin": {
 "type": "string",
 "description": "Origin city"
 },
 "destination": {
 "type": "string",
 "description": "Destination city"
 },
 "date": {
 "type": "string",
 "description": "Date in YYYY-MM-DD format"
 }
 },
 "required": ["origin", "destination", "date"]
 }
 }
}
]

```

# Implement tool functions

```

def get_weather(location: str, unit: str = "fahrenheit") -> Dict:
 """Get weather (mock)"""
 return {
 "location": location,
 "temperature": 72 if unit == "fahrenheit" else 22,
 "unit": unit,
 "condition": "sunny"
 }

```

```

def search_flights(origin: str, destination: str, date: str) -> List[Dict]:
 """Search flights (mock)"""

```

```

return [
 {
 "airline": "United",
 "flight_number": "UA123",
 "departure": f"{date} 08:00",
 "arrival": f"{date} 11:00",
 "price": 250
 },
 {
 "airline": "Delta",
 "flight_number": "DL456",
 "departure": f"{date} 14:00",
 "arrival": f"{date} 17:00",
 "price": 220
 }
]

Create agent and register tools
agent = FunctionCallingAgent(tools)
agent.register_tool("get_weather", get_weather)
agent.register_tool("search_flights", search_flights)

Usage
result = agent.run("What's the weather in San Francisco and find me flights from NYC to SFO")

print(f"Answer: {result['answer']}")
print(f"Iterations: {result['iterations']}")

Output:
[Agent calls get_weather(location="San Francisco, CA")]
[Agent calls search_flights(origin="NYC", destination="SFO", date="2025-03-15")]
Answer: The weather in San Francisco is sunny and 72°F. Here are flights from NYC to SFO:
- United UA123: Departs 8:00 AM, arrives 11:00 AM, $250
- Delta DL456: Departs 2:00 PM, arrives 5:00 PM, $220
The Delta flight is cheaper.
Iterations: 3

```

### Advantages over string parsing:

-  Structured output (JSON schema enforced)
-  Type safety (parameters validated)
-  No parsing errors
-  Parallel tool calls (if LLM decides)

### Disadvantages:

- ❌ Only works with OpenAI/Anthropic (not local models)
- ❌ Less control over tool selection logic

### Case 3: Supervised Agent (Human-in-the-Loop)

*Problem:* Fully autonomous agents can make mistakes with real-world consequences.

*Solution:* Require human approval for critical actions.

python

```
agents/supervised_agent.py
from enum import Enum
from typing import Dict, List, Callable, Optional

class ActionRisk(Enum):
 LOW = "low" # Auto-execute
 MEDIUM = "medium" # Log but auto-execute
 HIGH = "high" # Require approval
 CRITICAL = "critical" # Require approval + confirmation

class SupervisedAgent:
 def __init__(self, tools: List[Tool]):
 self.tools = {tool.name: tool for tool in tools}
 self.approval_required_actions = set()
 self.approval_callback = None

 def set_approval_callback(self, callback: Callable):
 """Set callback for human approval"""
 self.approval_callback = callback

 def mark_tool_as_high_risk(self, tool_name: str, risk_level: ActionRisk):
 """Mark tool as requiring approval"""
 if risk_level in [ActionRisk.HIGH, ActionRisk.CRITICAL]:
 self.approval_required_actions.add(tool_name)

 def run(self, query: str) -> Dict:
 """Run agent with human oversight"""

 state = {"query": query, "history": []}

 while not self._is_complete(state):
 # Generate action
 action = self._generate_action(state)
```

```

 # Check if approval required
 if action.tool in self.approval_required_actions:
 approved = self._request_approval(action)

 if not approved:
 # Human rejected action
 observation = Observation(
 result=None,
 success=False,
 error="Action rejected by human supervisor"
)
 state["history"].append({
 "action": action,
 "observation": observation,
 "approved": False
 })
 continue # Agent will try different approach

 # Execute action
 observation = self._execute_action(action)
 state["history"].append({
 "action": action,
 "observation": observation,
 "approved": True
 })

 if action.tool == "final_answer":
 break

 return {
 "answer": state["history"][-1]["observation"].result,
 "history": state["history"]
 }

def _request_approval(self, action: Action) -> bool:
 """Request human approval for action"""

 if self.approval_callback is None:
 # No approval mechanism set up, default to approve
 return True

 # Get human approval
 approval_request = {
 "tool": action.tool,
 "parameters": action.parameters,

```

```

 "reasoning": action.reasoning,
 "risk": self._assess_risk(action)
 }

 return self.approval_callback(approval_request)

def _assess_risk(self, action: Action) -> ActionRisk:
 """Assess risk level of action"""

 # Example risk assessment logic
 if action.tool in ["send_email", "make_purchase", "delete_file"]:
 return ActionRisk.HIGH
 elif action.tool in ["search", "read_file"]:
 return ActionRisk.LOW
 else:
 return ActionRisk.MEDIUM

Human approval UI (simple CLI example)
def cli_approval_callback(approval_request: Dict) -> bool:
 """Simple CLI approval interface"""

 print("\n" + "="*80)
 print("APPROVAL REQUIRED")
 print("="*80)
 print(f"Tool: {approval_request['tool']}")
 print(f"Parameters: {json.dumps(approval_request['parameters'], indent=2)}")
 print(f"Reasoning: {approval_request['reasoning']}")
 print(f"Risk Level: {approval_request['risk'].value}")
 print("="*80)

 while True:
 response = input("Approve this action? (y/n): ").lower()
 if response in ['y', 'yes']:
 return True
 elif response in ['n', 'no']:
 return False
 else:
 print("Please enter 'y' or 'n'")

Create supervised agent
agent = SupervisedAgent(tools)
agent.set_approval_callback(cli_approval_callback)

Mark high-risk tools
agent.mark_tool_as_high_risk("send_email", ActionRisk.HIGH)
agent.mark_tool_as_high_risk("make_purchase", ActionRisk.CRITICAL)

```



```

Usage
result = agent.run("Send an email to john@example.com with the summary of our meeting")

Output:
=====
APPROVAL REQUIRED
=====
Tool: send_email
Parameters: {
"to": "john@example.com",
"subject": "Meeting Summary",
"body": "...
}
Reasoning: User requested to send email with meeting summary
Risk Level: high
=====
Approve this action? (y/n): y
#
 Email sent

```

### When to use supervised agents:

- Financial transactions
- Data deletion
- External communications
- System changes
- Regulatory compliance requirements

## Expert-Level Comprehension Tests

**Question 1:** Your agent keeps calling the same search tool 10 times with slightly different queries, never finding what it needs. What's wrong and how do you fix it?

Expected Answer

**Root causes:**

1. **Tool returns uninformative results** - Search returns "no results" but doesn't suggest alternatives - LLM doesn't know what to do with empty result
2. **Agent doesn't know when to give up** - No explicit stopping condition - LLM keeps trying variations hoping for different result
3. **Query formulation problem** - LLM formulates queries that are too specific - Needs to try broader queries

**Diagnosis:** python

```

Analyze the search pattern
def analyze_search_pattern(agent_history):
 searches = [
 step for step in agent_history
 if step["action"]["tool"] == "search"
]

 print(f"Total searches: {len(searches)}")

 # Check if queries are similar
 queries = [s["action"]["parameters"]["query"] for s in searches]

 from difflib import SequenceMatcher

 for i in range(len(queries)-1):
 similarity = SequenceMatcher(None, queries[i], queries[i+1]).ratio()
 print(f"Query {i} vs {i+1}: {similarity:.2f} similar")
 print(f" '{queries[i]}'")
 print(f" '{queries[i+1]}'")

 # Check results
 for i, search in enumerate(searches):
 print(f"\nSearch {i}: '{search['action']['parameters']['query']}'")
 print(f"Result: {search['observation'][:100]}...")

Example output:
Total searches: 10
Query 0 vs 1: 0.85 similar ← Problem! Queries too similar
'population of Paris France 2024'
'population Paris France current'
Query 1 vs 2: 0.90 similar
'population Paris France current'
'how many people live in Paris France'
...
All searches return: "No results found"

```

**\*\*Fixes:\*\*** **\*\*Fix 1: Improve tool output\*\*** python

```

def search_with_suggestions(query: str) -> Dict:
 """Search with helpful feedback"""

 results = actual_search(query)

 if not results:

```

```

 # Provide helpful suggestions
 return {
 "results": [],
 "message": "No results found.",
 "suggestions": [
 "Try a broader query",
 "Check spelling",
 "Try different keywords",
 "This information may not be available"
]
 }

 return {
 "results": results,
 "message": f"Found {len(results)} results"
 }

```

**\*\*Fix 2: Add explicit stopping condition\*\*** python

```

SYSTEM_PROMPT = """
...
Important: If you've searched for the same information 3 times without success:
1. The information may not exist
2. Try a completely different approach
3. Or acknowledge you cannot find the answer
"""

```

**\*\*Fix 3: Detect and intervene\*\*** python

```

def detect_repeated_searches(history, threshold=3):
 """Detect if agent is searching repeatedly"""

 recent_searches = [
 h for h in history[-threshold:]
 if h["action"]["tool"] == "search"
]

 if len(recent_searches) == threshold:
 # All recent actions are searches - intervene
 return True

 return False

In agent loop

```

```

if detect_repeated_searches(state["history"]):
 state["history"].append({
 "system_intervention": """
 You've searched multiple times without finding the answer.
 Either:
 1. Try a completely different tool or approach
 2. Acknowledge that the information cannot be found
 Do NOT search again with a similar query.
 """
 })

```

**\*\*Fix 4: Query broadening strategy\*\* python**

```

def broaden_query(original_query: str, attempt: int) -> str:
 """Progressively broaden query"""

 if attempt == 1:
 return original_query # Original query
 elif attempt == 2:
 # Remove year/date
 return original_query.replace("2024", "").replace("current", "")
 elif attempt == 3:
 # Use only key terms
 key_terms = extract_key_terms(original_query)
 return " ".join(key_terms)
 else:
 # Give up
 return None

```

**\*\*Prevention:\*\* python**

```

Add to agent evaluation
def evaluate_search_efficiency(agent_runs):
 """Evaluate if agent searches efficiently"""

 metrics = []

 for run in agent_runs:
 searches = [s for s in run["history"] if s["action"]["tool"] == "search"]

 if len(searches) > 5:
 # Too many searches
 metrics.append({
 "query": run["query"],

```

```

 "num_searches": len(searches),
 "success": run["success"],
 "issue": "excessive_searching"
 })

 return metrics

```

**Question 2:** You need an agent that can: search web, read PDFs, use calculator, and send emails. How do you design the tool schemas to minimize agent confusion?

Expected Answer

**Principles for clear tool design:**

- 1. Distinct purposes** - Each tool should do ONE thing well
- 2. Clear descriptions** - Explain WHEN to use each tool
- 3. Concrete examples** - Show typical usage
- 4. Error handling** - Clear error messages
- 5. Composability** - Tools can work together

**Good tool schema design:** python

```

tools = [
 {
 "type": "function",
 "function": {
 "name": "web_search",
 "description": """Search the web for current information.

```

Use this when you need:

- Current events or news
- Real-time data (stock prices, weather, etc.)
- Information not in uploaded documents

Do NOT use this for:

- Information that might be in uploaded PDFs (use read\_pdf instead)
- Mathematical calculations (use calculator instead)

Examples:

- "What's the weather in Paris today?"
- "Who won the latest Super Bowl?"
- "What is the current price of AAPL stock?"

```

""",
 {
 "parameters": {
 "type": "object",
 "properties": {
 "query": {
 "type": "string",
 "description": "Search query (2-10 words work best)"
 }
 }
 }
 }
]

```

```

 },
 "required": ["query"]
 }
}
},
{
 "type": "function",
 "function": {
 "name": "read_pdf",
 "description": ""Read and extract information from uploaded PDF files.

```

Use this when you need:

- Information from documents the user uploaded
- Specific sections or pages from PDFs
- Text content from any uploaded file

Do NOT use this for:

- Current information (use web\_search instead)
- Information not in uploaded documents

Examples:

- "What does the contract say about termination?"
- "Summarize page 5 of the report"
- "Find all mentions of 'revenue' in quarterly\_report.pdf"

```

""",
 "parameters": {
 "type": "object",
 "properties": {
 "filename": {
 "type": "string",
 "description": "Name of the PDF file to read (must be uploaded)"
 },
 "query": {
 "type": "string",
 "description": "What information to extract from the PDF"
 },
 "pages": {
 "type": "array",
 "items": {"type": "integer"},
 "description": "Optional: specific pages to read (e.g., [1, 2, 5])"
 }
 },
 "required": ["filename", "query"]
 }
}
},

```

```
{
 "type": "function",
 "function": {
 "name": "calculator",
 "description": """Perform mathematical calculations.
```

Use this for:

- Arithmetic operations (+, -, \*, /)
- Percentages and ratios
- Compound calculations
- Financial calculations (interest, ROI, etc.)

Do NOT use for:

- Looking up numbers (use web\_search or read\_pdf)
- Statistical analysis beyond basic math

Examples:

- "What is 15% of 200?"
- "If I invest \$10,000 at 7% annual return for 20 years, what's the final amount?"
- "(500 \* 1.2) + 100"

Important: Use Python syntax. Examples:

- Exponentiation: 2\*\*3 (not 2^3)
- Division: 10/3 (not 10÷3)

```
""",
 "parameters": {
 "type": "object",
 "properties": {
 "expression": {
 "type": "string",
 "description": "Mathematical expression in Python syntax"
 }
 }
 },
 "required": ["expression"]
 }
},
{
 "type": "function",
 "function": {
 "name": "send_email",
 "description": """Send an email to specified recipients.
```

 HIGH RISK: This action sends real emails. Use with caution.

Use this when:

- User explicitly asks to send an email
- You have all required information (recipient, subject, body)

Do NOT use this:

- Without user confirmation
- To send to multiple recipients without explicit permission
- For sensitive information without encryption

Before calling, confirm with user:

- Recipient email addresses are correct
- Subject and body are accurate
- Timing is appropriate

Examples:

- "Send email to john@example.com with meeting notes"
- "Email the report to my manager at boss@company.com"

```

"""
 "parameters": {
 "type": "object",
 "properties": {
 "to": {
 "type": "array",
 "items": {"type": "string"},
 "description": "List of recipient email addresses"
 },
 "subject": {
 "type": "string",
 "description": "Email subject line"
 },
 "body": {
 "type": "string",
 "description": "Email body content"
 },
 "attachments": {
 "type": "array",
 "items": {"type": "string"},
 "description": "Optional: filenames of attachments"
 }
 },
 "required": ["to", "subject", "body"]
 }
}
]

```



**\*\*Additional best practices:\*\*** python

```
1. Tool categorization
READONLY_TOOLS = ["web_search", "read_pdf", "calculator"]
WRITE_TOOLS = ["send_email", "create_file", "delete_file"]

2. Tool dependencies
TOOL_DEPENDENCIES = {
 "send_email": {
 "requires": ["web_search", "read_pdf"], # Might need these first
 "conflicts": [] # Cannot be used with certain tools
 }
}

3. Tool usage examples in system prompt
SYSTEM_PROMPT = """
Example task: "Calculate the ROI from the investment report and email it to my boss"

Good execution:
1. read_pdf(filename="investment_report.pdf", query="initial investment and current value")
2. calculator(expression="(current_value - initial_investment) / initial_investment * 100")
3. send_email(to=["boss@company.com"], subject="Investment ROI", body="ROI is X%")

Bad execution:
1. web_search(query="investment ROI") ❌ Info is in PDF, not web!
2. send_email(...) ❌ Didn't calculate first!
"""

4. Tool validation
def validate_tool_call(tool_name: str, parameters: Dict, context: Dict) -> Optional[str]:
 """Validate tool call before execution"""

 if tool_name == "read_pdf":
 # Check if file exists
 if parameters["filename"] not in context["uploaded_files"]:
 return f"Error: File '{parameters['filename']}' not found. Available files: {context['uploaded_files']}"

 if tool_name == "send_email":
 # Check if email addresses are valid
 import re
 email_pattern = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'

 for email in parameters["to"]:
 if not re.match(email_pattern, email):
 return f"Error: Invalid email address: {email}"
```

```

if tool_name == "calculator":
 # Check if expression is safe
 dangerous_keywords = ["import", "exec", "eval", "open", "file"]
 if any(keyword in parameters["expression"].lower() for keyword in dangerous_keywords):
 return f"Error: Expression contains forbidden keywords"

return None # Validation passed

```

**\*\*Key insights:\*\*** - **\*\*Descriptions matter more than you think\*\*** - LLMs heavily weight tool descriptions - **\*\*Examples prevent 90% of misuse\*\*** - Show typical usage patterns - **\*\*Validation catches the rest\*\*** - Check parameters before execution - **\*\*Clear boundaries\*\*** - Explicitly state when NOT to use each tool

**Question 3:** Your agent successfully completes 85% of tasks but uses an average of 12 iterations when optimal would be 5. How do you improve efficiency without hurting success rate?

Expected Answer

**\*\*Diagnosis approach:\*\*** python

```

1. Analyze agent traces
def analyze_efficiency(agent_runs):
 """Identify efficiency issues"""

 issues = {
 'redundant_searches': [],
 'unnecessary_steps': [],
 'backtracking': [],
 'poor_planning': []
 }

 for run in agent_runs:
 history = run["history"]

 # Check for redundant searches
 searches = [h for h in history if h["action"]["tool"] == "search"]
 if len(searches) > 3:
 # Check if searching for same thing
 queries = [s["action"]["parameters"]["query"] for s in searches]
 if len(set(queries)) < len(queries) * 0.5:
 issues['redundant_searches'].append(run)

 # Check for backtracking (revisiting same tool)
 tools_used = [h["action"]["tool"] for h in history]
 tool_pattern = " ".join(tools_used)

```

```

 # Pattern like "search calc search calc" suggests poor planning
 if "search" in tool_pattern and tool_pattern.count("search") > tool_pattern.count("calc"):
 issues['backtracking'].append(run)

 # Check if final answer could have been reached earlier
 # (retrospective analysis)
 if run["success"]:
 optimal_path = find_optimal_path(history)
 if len(history) > len(optimal_path) * 1.5:
 issues['poor_planning'].append({
 'run': run,
 'actual_steps': len(history),
 'optimal_steps': len(optimal_path)
 })

 return issues

2. Find patterns
efficiency_issues = analyze_efficiency(production_agent_runs)

print(f"Redundant searches: {len(efficiency_issues['redundant_searches'])}")
print(f"Poor planning: {len(efficiency_issues['poor_planning'])}")

```

**\*\*Improvement strategies:\*\*** **\*\*Strategy 1: Add planning phase\*\*** python

```

SYSTEM_PROMPT = """
Before taking any actions, create a plan:

1. Understand the goal
2. Identify what information you need
3. Plan the sequence of tools to use
4. Execute the plan step by step

Example:
Goal: "Find the cheapest flight from NYC to LA next week and calculate the cost per mile"

Plan:
1. Search for flights (web_search)
2. Find cheapest option from results
3. Calculate miles between NYC and LA (web_search or calculator with formula)
4. Divide cost by miles (calculator)
5. Return answer

Now execute:

```

```

Step 1: web_search(...)
...
"""

This reduces average from 12 → 8 iterations (33% improvement)

```

**\*\*Strategy 2: Tool result caching\*\* python**

```

class CachingAgent(ReActAgent):
 def __init__(self, *args, **kwargs):
 super().__init__(*args, **kwargs)
 self.tool_cache = {}

 def _execute_action(self, action: Action) -> Observation:
 """Execute with caching"""

 # Create cache key
 cache_key = (action.tool, json.dumps(action.parameters, sort_keys=True))

 # Check cache
 if cache_key in self.tool_cache:
 print(f"[CACHE HIT] {action.tool}({action.parameters})")
 return self.tool_cache[cache_key]

 # Execute
 observation = super()._execute_action(action)

 # Cache if successful
 if observation.success:
 self.tool_cache[cache_key] = observation

 return observation

This prevents redundant tool calls
Reduces iterations by 20-30% when agent backtracks

```

**\*\*Strategy 3: Few-shot examples of efficient execution\*\* python**

```

SYSTEM_PROMPT = """
Here are examples of EFFICIENT task completion:

Task: "What's 15% tip on a $80 meal?"
❌ Inefficient (3 steps):
1. web_search("how to calculate tip")

```

```
2. web_search("15 percent of 80")
3. calculator("80 * 0.15")
```

✓ Efficient (1 step):

```
1. calculator("80 * 0.15")
Result: $12
```

Task: "Find the population of Paris and calculate how many buses are needed if each holds 50 people"

✗ Inefficient (5 steps):

```
1. web_search("Paris France")
2. web_search("population of Paris")
3. web_search("how many buses needed")
4. calculator("2161000") # Invalid
5. calculator("2161000 / 50")
```

✓ Efficient (2 steps):

```
1. web_search("population of Paris France")
Result: 2.161 million
2. calculator("2161000 / 50")
Result: 43,220 buses
"""
```

**\*\*Strategy 4: Add efficiency metric to agent prompt\*\*** python

```
SYSTEM_PROMPT = """
You are an efficient assistant. Your goal is to complete tasks in the FEWEST steps possible.

Current efficiency:
- Steps taken: {current_iteration}
- Target: < 5 steps for most tasks

Tips for efficiency:
1. Don't search for information you already have
2. Combine calculations when possible
3. Plan before acting
4. If you've taken more than 5 steps, consider if you're overthinking
"""

Update prompt each iteration
def _build_react_prompt(self, state: Dict) -> str:
 prompt = base_prompt

 # Add efficiency warning if needed
 if state["iteration"] > 5:
 prompt += "\n⚠ You've taken 5+ steps. Can you complete the task now?"
```

```
return prompt
```

**\*\*Strategy 5: Evaluate and optimize\*\*** python

```
Create efficiency benchmark
efficiency_test_cases = [
 {
 "query": "What is 15% of 200?",
 "optimal_steps": 1, # calculator("200 * 0.15")
 "max_acceptable_steps": 2
 },
 {
 "query": "Find the population of Tokyo and Paris, then calculate the difference"
 "optimal_steps": 3, # search(Tokyo), search(Paris), calc(difference)
 "max_acceptable_steps": 5
 }
]

def evaluate_efficiency(agent, test_cases):
 """Measure efficiency"""

 results = []

 for test in test_cases:
 result = agent.run(test["query"])

 efficiency_score = test["optimal_steps"] / result["iterations"]
 passed = result["iterations"] <= test["max_acceptable_steps"]

 results.append({
 "query": test["query"],
 "optimal": test["optimal_steps"],
 "actual": result["iterations"],
 "efficiency": efficiency_score,
 "passed": passed
 })

 avg_efficiency = np.mean([r["efficiency"] for r in results])
 pass_rate = np.mean([r["passed"] for r in results])

 return {
 "avg_efficiency": avg_efficiency,
 "pass_rate": pass_rate,
 "results": results
 }
```

```

 }

 # Iterate until efficiency improves
 baseline_efficiency = evaluate_efficiency(agent_v1, efficiency_test_cases)
 improved_efficiency = evaluate_efficiency(agent_v2_with_planning, efficiency_test_cases)

 print(f"Baseline: {baseline_efficiency['avg_efficiency']:.2f}")
 print(f"Improved: {improved_efficiency['avg_efficiency']:.2f}")
 print(f"Improvement: {(improved_efficiency['avg_efficiency'] / baseline_efficiency['avg_efficiency']):.2f}")

```

**\*\*Expected results:\*\*** - Baseline: 12 iterations, 85% success - With planning: 8 iterations, 85% success (33% efficiency gain) - With caching: 6 iterations, 87% success (50% efficiency gain, slight quality improvement) - With few-shot: 5 iterations, 86% success (58% efficiency gain)

**\*\*Key insight:\*\*** Efficiency improvements often IMPROVE success rate because: - Fewer steps = fewer opportunities for errors - Clearer reasoning = better decisions - Lower cost = can use better models

---

### 3. Fastest Path to Competence (Execution-Focused)

---

#### Prerequisites (Week 0: Setup)

**Install:**

bash

```

pip install \
 openai anthropic \ # LLM APIs
 langchain \ # Agent framework (optional, will build from scratch first)
 langchain-openai \ # OpenAI integration
 langgraph \ # Graph-based agent orchestration
 beautifulsoup4 \ # Web scraping for tools
 requests \ # HTTP requests
 python-dotenv # Environment variables

```

**Repository structure:**

```

agent-systems/
├── agents/
│ ├── react_agent.py # ReAct from scratch
│ └── function_calling_agent.py # OpenAI function calling

```

```

| ├── supervised_agent.py # Human-in-loop
| └── langchain_agent.py # Using LangChain
├── tools/
| ├── search.py # Web search tool
| ├── calculator.py # Math tool
| ├── file_tools.py # File read/write
| └── custom_tools.py # Domain-specific
├── evaluation/
| ├── agent_metrics.py # Success rate, efficiency
| └── test_cases.py # Agent test suite
├── examples/
| ├── research_assistant.py # Research agent
| ├── data_analyst.py # Analysis agent
| └── customer_support.py # Support agent
└── README.md

```

---

## Week 1: ReAct Agent from Scratch

### Learning Objectives:

- Understand ReAct pattern deeply
- Implement agent loop with control
- Build tool interfaces
- Debug agent failures

### Day 1-3: Core ReAct Implementation

(Already covered in Case 1 above, but here's the complete hands-on lab)

python

```

agents/react_agent.py
(Full implementation from Case 1)

Additional: Tool implementations
tools/calculator.py
import ast
import operator

class SafeCalculator:
 """Safe mathematical expression evaluator"""

```



```

Allowed operations
operators = {
 ast.Add: operator.add,
 ast.Sub: operator.sub,
 ast.Mult: operator.mul,
 ast.Div: operator.truediv,
 ast.Pow: operator.pow,
 ast.USub: operator.neg
}

def evaluate(self, expression: str) -> float:
 """Safely evaluate mathematical expression"""
 try:
 tree = ast.parse(expression, mode='eval')
 return self._eval_node(tree.body)
 except Exception as e:
 raise ValueError(f"Invalid expression: {e}")

def _eval_node(self, node):
 """Recursively evaluate AST nodes"""
 if isinstance(node, ast.Num):
 return node.n
 elif isinstance(node, ast.BinOp):
 left = self._eval_node(node.left)
 right = self._eval_node(node.right)
 op = self.operators.get(type(node.op))
 if op is None:
 raise ValueError(f"Unsupported operation: {type(node.op)}")
 return op(left, right)
 elif isinstance(node, ast.UnaryOp):
 operand = self._eval_node(node.operand)
 op = self.operators.get(type(node.op))
 if op is None:
 raise ValueError(f"Unsupported operation: {type(node.op)}")
 return op(operand)
 else:
 raise ValueError(f"Unsupported expression: {type(node)}")

Usage
calc = SafeCalculator()
print(calc.evaluate("2 + 2")) # 4
print(calc.evaluate("10 * (5 + 3)")) # 80
print(calc.evaluate("2 ** 10")) # 1024

```

```

tools/search.py
import requests
from typing import List, Dict

class WebSearchTool:
 """Web search using SerpAPI or similar"""

 def __init__(self, api_key: str = None):
 self.api_key = api_key or os.getenv("SERPAPI_KEY")

 def search(self, query: str, num_results: int = 5) -> List[Dict]:
 """Search the web"""

 # Using SerpAPI (requires API key)
 if self.api_key:
 url = "https://serpapi.com/search"
 params = {
 "q": query,
 "api_key": self.api_key,
 "num": num_results
 }

 response = requests.get(url, params=params)
 data = response.json()

 results = []
 for result in data.get("organic_results", []):
 results.append({
 "title": result.get("title"),
 "snippet": result.get("snippet"),
 "link": result.get("link")
 })

 return results
 else:
 # Mock implementation
 return [{
 "title": f"Result for: {query}",
 "snippet": f"This is a mock search result for '{query}'",
 "link": "https://example.com"
 }]

 def format_results(self, results: List[Dict]) -> str:
 """Format search results for LLM"""
 if not results:

```

```

 return "No results found."

 formatted = []
 for i, result in enumerate(results, 1):
 formatted.append(f"{i}. {result['title']}")
 formatted.append(f" {result['snippet']}")
 formatted.append(f" URL: {result['link']}")

 return "\n".join(formatted)

```

**Deliverable:** Working ReAct agent that:

- Implements full ReAct loop
- Has calculator and search tools
- Handles errors gracefully
- Detects infinite loops

## Day 4-5: Control Mechanisms

python

```

agents/controlled_agent.py
from typing import Dict, Optional
import time

class ControlledReActAgent(ReActAgent):
 """ReAct agent with comprehensive control mechanisms"""

 def __init__(self, tools: List[Tool],
 max_iterations: int = 10,
 max_cost: float = 1.0,
 timeout: int = 60):
 super().__init__(tools, max_iterations)
 self.max_cost = max_cost
 self.timeout = timeout

 def run(self, query: str) -> Dict:
 """Run agent with multiple control mechanisms"""

 state = {
 "query": query,
 "scratchpad": [],
 "iteration": 0,
 "cost": 0.0,

```

```

 "start_time": time.time()
 }

 while True:
 # Check iteration limit
 if state["iteration"] >= self.max_iterations:
 return self._handle_max_iterations(state)

 # Check cost limit
 if state["cost"] >= self.max_cost:
 return self._handle_budget_exceeded(state)

 # Check timeout
 elapsed = time.time() - state["start_time"]
 if elapsed >= self.timeout:
 return self._handle_timeout(state)

 # Check for loops
 if self._detect_loop(state["scratchpad"]):
 return self._handle_loop_detected(state)

 # Generate and execute action
 state["iteration"] += 1
 action = self._generate_action(state)

 # Track cost
 state["cost"] += self._estimate_cost(action)

 # Execute
 if action.tool == "final_answer":
 return {
 "answer": action.parameters["answer"],
 "success": True,
 "iterations": state["iteration"],
 "cost": state["cost"],
 "time": elapsed
 }

 observation = self._execute_action(action)
 state["scratchpad"].append({
 "thought": action.reasoning,
 "action": action.tool,
 "parameters": action.parameters,
 "observation": observation
 })

```

```

def _handle_max_iterations(self, state: Dict) -> Dict:
 """Handle max iterations exceeded"""
 return {
 "error": "max_iterations_exceeded",
 "message": f"Reached maximum of {self.max_iterations} iterations",
 "partial_result": self._extract_partial_result(state),
 "iterations": state["iteration"],
 "cost": state["cost"]
 }

def _handle_budget_exceeded(self, state: Dict) -> Dict:
 """Handle budget exceeded"""
 return {
 "error": "budget_exceeded",
 "message": f"Cost ${state['cost']:.2f} exceeded budget ${self.max_cost:.2f}",
 "partial_result": self._extract_partial_result(state),
 "iterations": state["iteration"],
 "cost": state["cost"]
 }

def _handle_timeout(self, state: Dict) -> Dict:
 """Handle timeout"""
 elapsed = time.time() - state["start_time"]
 return {
 "error": "timeout",
 "message": f"Execution time {elapsed:.1f}s exceeded timeout {self.timeout}s",
 "partial_result": self._extract_partial_result(state),
 "iterations": state["iteration"],
 "cost": state["cost"]
 }

def _handle_loop_detected(self, state: Dict) -> Dict:
 """Handle infinite loop"""
 return {
 "error": "infinite_loop_detected",
 "message": "Agent appears stuck in a loop",
 "loop_pattern": self._analyze_loop(state["scratchpad"]),
 "partial_result": self._extract_partial_result(state),
 "iterations": state["iteration"],
 "cost": state["cost"]
 }

def _extract_partial_result(self, state: Dict) -> str:
 """Extract best available partial answer"""
 # Look through scratchpad for any useful information
 # This helps users get SOMETHING even when agent fails

```

```

 if not state["scratchpad"]:
 return "No progress made"

 # Get last few observations
 recent_observations = [
 step["observation"] for step in state["scratchpad"][-3:]
]

 return f"Partial information gathered: {' '.join(str(obs) for obs in recent_observations)}"

 def _estimate_cost(self, action: Action) -> float:
 """Estimate cost of action"""
 # Rough estimation based on typical token usage
 if action.tool == "search":
 return 0.001 # Search is cheap
 elif action.tool == "final_answer":
 return 0.0
 else:
 return 0.005 # Other tools

Usage
agent = ControlledReActAgent(
 tools=[search_tool, calculator_tool],
 max_iterations=10,
 max_cost=0.50,
 timeout=30
)

result = agent.run("Complex query that might cause issues")

if "error" in result:
 print(f"Agent failed: {result['error']}")
 print(f"Message: {result['message']}")
 print(f"Partial result: {result['partial_result']}")
else:
 print(f"Success: {result['answer']}")
 print(f"Used {result['iterations']} iterations, ${result['cost']:.3f}")

```

**Deliverable:** Controlled agent with:

- Max iterations
- Budget limits
- Timeout handling

- Loop detection
- Partial results on failure

## Day 6-7: Agent Debugging Tools

python

```
evaluation/agent_debugger.py
import json
from typing import Dict, List
import pandas as pd

class AgentDebugger:
 """Tools for debugging agent behavior"""

 def visualize_execution(self, agent_result: Dict):
 """Visualize agent execution trace"""

 print("AGENT EXECUTION TRACE")
 print("=" * 80)
 print(f"Query: {agent_result['query']}")
 print(f"Result: {'✅ Success' if agent_result.get('success') else '❌ Failed'}")
 print(f"Iterations: {agent_result['iterations']}")
 print(f"Cost: ${agent_result.get('cost', 0):.3f}")
 print("=" * 80)

 for i, step in enumerate(agent_result.get('scratchpad', []), 1):
 print(f"\nStep {i}:")
 print(f" Thought: {step['thought']}")
 print(f" Action: {step['action']}({json.dumps(step['parameters'])})")
 print(f" Observation: {str(step['observation'])[:200]}")

 def analyze_failures(self, agent_runs: List[Dict]) -> Dict:
 """Analyze common failure patterns"""

 failures = [run for run in agent_runs if not run.get('success')]

 if not failures:
 return {"message": "No failures to analyze"}

 # Categorize failures
 error_types = {}
 for failure in failures:
 error = failure.get('error', 'unknown')
 if error not in error_types:
```

```

 error_types[error] = []
 error_types[error].append(failure)

Analyze patterns
analysis = {}

for error_type, examples in error_types.items():
 avg_iterations = np.mean([e['iterations'] for e in examples])

 # Get common tools used before failure
 tools_used = []
 for example in examples:
 for step in example.get('scratchpad', []):
 tools_used.append(step['action'])

 from collections import Counter
 common_tools = Counter(tools_used).most_common(3)

 analysis[error_type] = {
 'count': len(examples),
 'percentage': len(examples) / len(failures) * 100,
 'avg_iterations': avg_iterations,
 'common_tools': common_tools,
 'examples': [e['query'] for e in examples[:3]]
 }

return analysis

def compare_agents(self, agent_a_results: List[Dict],
 agent_b_results: List[Dict]) -> pd.DataFrame:
 """Compare two agent versions"""

 def aggregate_metrics(results):
 return {
 'success_rate': np.mean([r.get('success', False) for r in results]),
 'avg_iterations': np.mean([r['iterations'] for r in results]),
 'avg_cost': np.mean([r.get('cost', 0) for r in results]),
 'avg_time': np.mean([r.get('time', 0) for r in results])
 }

 metrics_a = aggregate_metrics(agent_a_results)
 metrics_b = aggregate_metrics(agent_b_results)

 comparison = pd.DataFrame({
 'Agent A': metrics_a,
 'Agent B': metrics_b
 })

```



```

 })

 comparison['Improvement'] = (
 (comparison['Agent B'] - comparison['Agent A']) / comparison['Agent A'] * 100
)

 return comparison

Usage
debugger = AgentDebugger()

Visualize single execution
result = agent.run("What is 15% of 200?")
debugger.visualize_execution(result)

Analyze failures across many runs
all_runs = [agent.run(query) for query in test_queries]
failure_analysis = debugger.analyze_failures(all_runs)

print("\nFailure Analysis:")
for error_type, stats in failure_analysis.items():
 print(f"\n{error_type}: {stats['count']} ({stats['percentage']:.1f}%)")
 print(f" Avg iterations: {stats['avg_iterations']:.1f}")
 print(f" Common tools: {stats['common_tools']}")
 print(f" Examples: {stats['examples']}")

Compare agent versions
agent_v1_results = [agent_v1.run(q) for q in test_queries]
agent_v2_results = [agent_v2.run(q) for q in test_queries]

comparison = debugger.compare_agents(agent_v1_results, agent_v2_results)
print("\nAgent Comparison:")
print(comparison)

```

**Deliverable:** Debugging toolkit with:

- Execution visualization
  - Failure pattern analysis
  - Agent comparison tools
-

## Week 2: Production Patterns & Tool Design

### Learning Objectives:

- Design effective tool schemas
- Implement function calling
- Build domain-specific agents
- Handle tool errors

### Day 8-10: Tool Design Best Practices

python

```
tools/tool_designer.py
from typing import Dict, List, Any, Callable
from pydantic import BaseModel, Field
import json

class ToolParameter(BaseModel):
 """Parameter specification for a tool"""
 name: str
 type: str # "string", "number", "boolean", "array", "object"
 description: str
 required: bool = True
 enum: List[str] = None
 default: Any = None

class ToolSchema(BaseModel):
 """Schema for a well-designed tool"""
 name: str = Field(..., description="Tool name (snake_case, descriptive)")
 description: str = Field(..., description="What the tool does and when to use it")
 parameters: List[ToolParameter]
 returns: Dict[str, Any] = Field(..., description="What the tool returns")
 examples: List[Dict[str, Any]] = Field(..., description="Example usages")
 error_handling: str = Field(..., description="How errors are communicated")

 def to_openai_function(self) -> Dict:
 """Convert to OpenAI function calling format"""

 properties = {}
 required = []

 for param in self.parameters:
 prop = {
```

```

 "type": param.type,
 "description": param.description
 }

 if param.enum:
 prop["enum"] = param.enum

 properties[param.name] = prop

 if param.required:
 required.append(param.name)

 return {
 "type": "function",
 "function": {
 "name": self.name,
 "description": self.description,
 "parameters": {
 "type": "object",
 "properties": properties,
 "required": required
 }
 }
 }
}

```

```

class ToolDesigner:
 """Helper for designing effective tools"""

 @staticmethod
 def validate_tool_schema(schema: ToolSchema) -> List[str]:
 """Validate tool design against best practices"""

 issues = []

 # Check name
 if not schema.name.islower() or ' ' in schema.name:
 issues.append("Tool name should be lowercase snake_case")

 # Check description length
 if len(schema.description) < 50:
 issues.append("Description too short (recommend 50+ chars)")

 if len(schema.description) > 500:
 issues.append("Description too long (recommend <500 chars)")

 # Check for "when to use" guidance

```

```

 if "use this" not in schema.description.lower():
 issues.append("Description should include 'Use this when...' guidance")

 # Check for examples
 if not schema.examples or len(schema.examples) < 2:
 issues.append("Should include at least 2 examples")

 # Check parameters
 if not schema.parameters:
 issues.append("Tool should have at least one parameter")

 for param in schema.parameters:
 if len(param.description) < 10:
 issues.append(f"Parameter '{param.name}' description too short")

 return issues

```

```

@staticmethod
def create_example_tool() -> ToolSchema:
 """Example of well-designed tool"""

 return ToolSchema(
 name="search_database",
 description="""Search a database for records matching criteria.

```

Use this when you need to:

- Find specific records in the database
- Filter records by multiple criteria
- Get counts or aggregates

Do NOT use this for:

- Web searches (use web\_search instead)
- Mathematical calculations (use calculator)

Best practices:

- Be specific with filters
- Use date ranges when searching by date
- Limit results to avoid overwhelming output""",

```

 parameters=[
 ToolParameter(
 name="table",
 type="string",
 description="Database table to search (e.g., 'customers', 'orders',
 required=True
),

```

```

 ToolParameter(
 name="filters",
 type="object",
 description="Filter criteria as key-value pairs (e.g., {'status': 'a",
 required=False
),
 ToolParameter(
 name="limit",
 type="number",
 description="Maximum number of results to return (default: 10, max:",
 required=False,
 default=10
),
 ToolParameter(
 name="sort_by",
 type="string",
 description="Field to sort results by (prefix with '-' for descending",
 required=False
)
],

 returns={
 "type": "object",
 "properties": {
 "results": {"type": "array", "description": "Matching records"},
 "count": {"type": "number", "description": "Total count"},
 "has_more": {"type": "boolean", "description": "Whether more results"}
 }
 },

 examples=[
 {
 "description": "Find active customers in US",
 "input": {
 "table": "customers",
 "filters": {"status": "active", "country": "US"},
 "limit": 10
 },
 "output": {
 "results": [{"id": 1, "name": "Acme Corp", ...}],
 "count": 150,
 "has_more": True
 }
 },
 {
 "description": "Get recent orders sorted by date",

```

```

 "input": {
 "table": "orders",
 "filters": {"date": {"gte": "2025-01-01"}},
 "sort_by": "-date",
 "limit": 20
 },
 "output": {
 "results": [{"id": 501, "date": "2025-02-01", ...}],
 "count": 45,
 "has_more": True
 }
 }
],

 error_handling="""Returns error object with:
- error: Error type ('invalid_table', 'invalid_filters', 'query_failed')
- message: Human-readable error description
- suggestion: What to try instead"""
)

Usage
designer = ToolDesigner()

Create tool schema
tool_schema = designer.create_example_tool()

Validate
issues = designer.validate_tool_schema(tool_schema)
if issues:
 print("Tool design issues:")
 for issue in issues:
 print(f" - {issue}")
else:
 print("✅ Tool schema is well-designed")

Convert to OpenAI format
openai_function = tool_schema.to_openai_function()
print(json.dumps(openai_function, indent=2))

```

**Deliverable:** Tool design framework with:

- Schema definition
- Validation rules
- Best practices

- Example tools

## Day 11-13: Domain-Specific Agent

python

```
examples/data_analyst_agent.py
"""Data analyst agent for business intelligence"""

from typing import List, Dict
import pandas as pd

class DataAnalystAgent:
 """Agent that can analyze data and answer business questions"""

 def __init__(self):
 self.tools = self._create_tools()
 self.agent = FunctionCallingAgent(self.tools)
 self._register_tool_implementations()

 def _create_tools(self) -> List[Dict]:
 """Define tools for data analysis"""

 return [
 {
 "type": "function",
 "function": {
 "name": "load_dataset",
 "description": """Load a dataset into memory for analysis

Use this to:
- Load CSV, Excel, or JSON files
- Prepare data for analysis

Available datasets:
- sales_data.csv: Monthly sales by product and region
- customer_data.csv: Customer demographics and purchase history
- inventory_data.csv: Current inventory levels
""",
 "parameters": {
 "type": "object",
 "properties": {
 "filename": {
 "type": "string",
 "description": "Dataset filename"
 }
 }
 }
 }
 }
]
```

```

 }
 },
 "required": ["filename"]
}
},
{
 "type": "function",
 "function": {
 "name": "analyze_data",
 "description": ""Perform statistical analysis on loaded dataset.

```

Supported analyses:

- summary: Descriptive statistics (mean, median, std, etc.)
- correlation: Correlation matrix
- trend: Time series trend analysis
- groupby: Group data and aggregate
- distribution: Distribution statistics

```

""",
 "parameters": {
 "type": "object",
 "properties": {
 "analysis_type": {
 "type": "string",
 "enum": ["summary", "correlation", "trend", "groupby", "distribution"],
 "description": "Type of analysis to perform"
 },
 "columns": {
 "type": "array",
 "items": {"type": "string"},
 "description": "Columns to analyze"
 },
 "group_by": {
 "type": "string",
 "description": "Column to group by (for groupby analysis)"
 },
 "agg_function": {
 "type": "string",
 "enum": ["sum", "mean", "count", "min", "max"],
 "description": "Aggregation function (for groupby)"
 }
 },
 "required": ["analysis_type"]
 }
},
}
},

```



```

 {
 "type": "function",
 "function": {
 "name": "create_visualization",
 "description": """Create a visualization of the data.

Chart types:
- line: Time series or trend
- bar: Comparisons across categories
- scatter: Relationship between two variables
- histogram: Distribution of values
- pie: Composition/proportions
""",
 "parameters": {
 "type": "object",
 "properties": {
 "chart_type": {
 "type": "string",
 "enum": ["line", "bar", "scatter", "histogram", "pie"],
 "description": "Type of chart"
 },
 "x_column": {
 "type": "string",
 "description": "Column for x-axis"
 },
 "y_column": {
 "type": "string",
 "description": "Column for y-axis"
 },
 "title": {
 "type": "string",
 "description": "Chart title"
 }
 },
 "required": ["chart_type"]
 }
 }
 }
]

 def _register_tool_implementations(self):
 """Register tool implementations"""

 self.current_dataset = None

 self.agent.register_tool("load_dataset", self._load_dataset)

```

```

self.agent.register_tool("analyze_data", self._analyze_data)
self.agent.register_tool("create_visualization", self._create_visualization)

def _load_dataset(self, filename: str) -> Dict:
 """Load dataset"""
 try:
 if filename.endswith('.csv'):
 self.current_dataset = pd.read_csv(f"data/{filename}")
 elif filename.endswith('.xlsx'):
 self.current_dataset = pd.read_excel(f"data/{filename}")
 else:
 return {"error": f"Unsupported file format: {filename}"}

 return {
 "success": True,
 "rows": len(self.current_dataset),
 "columns": list(self.current_dataset.columns),
 "preview": self.current_dataset.head().to_dict()
 }
 except Exception as e:
 return {"error": str(e)}

def _analyze_data(self, analysis_type: str, columns: List[str] = None, **kwargs) -> Dict:
 """Perform analysis"""
 if self.current_dataset is None:
 return {"error": "No dataset loaded. Use load_dataset first."}

 try:
 if analysis_type == "summary":
 result = self.current_dataset[columns].describe().to_dict() if columns else self.current_dataset.describe().to_dict()

 elif analysis_type == "correlation":
 result = self.current_dataset[columns].corr().to_dict() if columns else self.current_dataset.corr().to_dict()

 elif analysis_type == "groupby":
 group_by = kwargs.get("group_by")
 agg_func = kwargs.get("agg_function", "sum")
 result = self.current_dataset.groupby(group_by)[columns].agg(agg_func).to_dict()

 else:
 return {"error": f"Unknown analysis type: {analysis_type}"}

 return {"success": True, "result": result}
 except Exception as e:
 return {"error": str(e)}

```

```

def _create_visualization(self, chart_type: str, **kwargs) -> Dict:
 """Create visualization"""
 if self.current_dataset is None:
 return {"error": "No dataset loaded"}

 # In production, would create actual chart
 return {
 "success": True,
 "chart_type": chart_type,
 "message": f"Created {chart_type} chart",
 "filename": f"{chart_type}_chart.png"
 }

def query(self, question: str) -> Dict:
 """Answer business question"""
 return self.agent.run(question)

Usage
analyst = DataAnalystAgent()

Example queries
result = analyst.query("""
Load the sales_data.csv and tell me:
1. What were the total sales by region?
2. Which product had the highest sales?
3. Create a bar chart showing sales by region.
""")

print(result['answer'])

```

**Deliverable:** Domain-specific agent demonstrating:

- Multiple specialized tools
- Domain knowledge in descriptions
- Error handling
- Practical use case

## Day 14: Tool Error Handling

python

```

tools/resilient_tools.py
from typing import Dict, Any, Callable

```

```

from functools import wraps
import time

class ToolError(Exception):
 """Base exception for tool errors"""
 pass

class ToolExecutionError(ToolError):
 """Tool execution failed"""
 pass

class ToolValidationError(ToolError):
 """Tool parameters invalid"""
 pass

class ToolTimeoutError(ToolError):
 """Tool execution timed out"""
 pass

def with_retries(max_retries: int = 3, backoff: float = 1.0):
 """Decorator to add retry logic to tools"""

 def decorator(func: Callable) -> Callable:
 @wraps(func)
 def wrapper(*args, **kwargs):
 last_error = None

 for attempt in range(max_retries):
 try:
 return func(*args, **kwargs)
 except ToolExecutionError as e:
 last_error = e
 if attempt < max_retries - 1:
 time.sleep(backoff * (2 ** attempt)) # Exponential backoff
 continue

 # All retries failed
 raise ToolExecutionError(f"Failed after {max_retries} attempts: {last_error}")

 return wrapper
 return decorator

def with_timeout(timeout_seconds: int = 30):
 """Decorator to add timeout to tools"""

 def decorator(func: Callable) -> Callable:

```

```

@wraps(func)
def wrapper(*args, **kwargs):
 import signal

 def timeout_handler(signum, frame):
 raise ToolTimeoutError(f"Tool execution exceeded {timeout_seconds}s")

 # Set timeout
 signal.signal(signal.SIGALRM, timeout_handler)
 signal.alarm(timeout_seconds)

 try:
 result = func(*args, **kwargs)
 signal.alarm(0) # Cancel timeout
 return result
 except ToolTimeoutError:
 raise
 finally:
 signal.alarm(0)

 return wrapper
return decorator

def with_validation(schema: Dict[str, Any]):
 """Decorator to validate tool parameters"""

 def decorator(func: Callable) -> Callable:
 @wraps(func)
 def wrapper(*args, **kwargs):
 # Validate required parameters
 for param_name, param_spec in schema.items():
 if param_spec.get("required", False) and param_name not in kwargs:
 raise ToolValidationError(f"Missing required parameter: {param_name}")

 # Type checking
 if param_name in kwargs:
 expected_type = param_spec.get("type")
 actual_value = kwargs[param_name]

 if expected_type == "string" and not isinstance(actual_value, str):
 raise ToolValidationError(f"Parameter '{param_name}' must be str")
 elif expected_type == "number" and not isinstance(actual_value, (int, float)):
 raise ToolValidationError(f"Parameter '{param_name}' must be number")

 # Enum validation
 if "enum" in param_spec and actual_value not in param_spec["enum"]:

```

```

 raise ToolValidationError(
 f"Parameter '{param_name}' must be one of: {param_spec['enum']}"
)

 return func(*args, **kwargs)

 return wrapper
return decorator

Example resilient tool
@with_validation({
 "url": {"type": "string", "required": True},
 "method": {"type": "string", "enum": ["GET", "POST"], "required": False}
})
@with_retries(max_retries=3, backoff=1.0)
@with_timeout(timeout_seconds=10)
def fetch_url(url: str, method: str = "GET") -> Dict:
 """Fetch URL with retries, timeout, and validation"""

 import requests

 try:
 if method == "GET":
 response = requests.get(url)
 else:
 response = requests.post(url)

 response.raise_for_status()

 return {
 "success": True,
 "status_code": response.status_code,
 "content": response.text[:1000] # Truncate
 }

 except requests.RequestException as e:
 raise ToolExecutionError(f"HTTP request failed: {e}")

Usage
try:
 result = fetch_url(url="https://example.com", method="GET")
 print(result)
except ToolValidationError as e:
 print(f"Validation error: {e}")
except ToolTimeoutError as e:
 print(f"Timeout: {e}")

```

```
except ToolExecutionError as e:
 print(f"Execution error: {e}")
```

**Deliverable:** Resilient tool implementation with:

- Automatic retries
  - Timeout handling
  - Parameter validation
  - Clear error messages
- 

## Week 3: Agent Evaluation & Production Deployment

**Learning Objectives:**

- Evaluate agent performance
- Deploy agent to production
- Monitor agent behavior
- Create portfolio documentation

### Day 15-17: Agent-Specific Evaluation

python

```
evaluation/agent_metrics.py
from typing import List, Dict
import numpy as np

class AgentEvaluator:
 """Evaluate agent performance"""

 def evaluate_agent(self, agent, test_cases: List[Dict]) -> Dict:
 """Comprehensive agent evaluation"""

 results = []

 for test_case in test_cases:
 result = agent.run(test_case["query"])

 # Evaluate this run
 evaluation = {
 "query": test_case["query"],
```

```

 "success": result.get("success", False),
 "iterations": result["iterations"],
 "cost": result.get("cost", 0),
 "time": result.get("time", 0),
 "correct": self._check_correctness(
 result.get("answer"),
 test_case.get("expected_answer")
),
 "tools_used": self._extract_tools_used(result),
 "error_type": result.get("error")
 }

 results.append(evaluation)

Aggregate metrics
aggregated = self._aggregate_metrics(results)

return {
 "individual_results": results,
 "aggregated": aggregated
}

def _check_correctness(self, answer: str, expected: str) -> bool:
 """Check if answer is correct"""
 if not answer or not expected:
 return False

 # Simple check (in production, use more sophisticated comparison)
 return expected.lower() in answer.lower()

def _extract_tools_used(self, result: Dict) -> List[str]:
 """Extract which tools were used"""
 tools = []
 for step in result.get("scratchpad", []):
 tools.append(step.get("action"))
 return tools

def _aggregate_metrics(self, results: List[Dict]) -> Dict:
 """Aggregate metrics across all test cases"""

 successful = [r for r in results if r["success"]]

 return {
 # Core metrics
 "success_rate": len(successful) / len(results) if results else 0,
 "correctness_rate": np.mean([r["correct"] for r in results]),
 }

```



```

 # Efficiency metrics
 "avg_iterations": np.mean([r["iterations"] for r in results]),
 "avg_iterations_successful": np.mean([r["iterations"] for r in successful])

 # Cost metrics
 "avg_cost": np.mean([r["cost"] for r in results]),
 "total_cost": sum(r["cost"] for r in results),

 # Latency metrics
 "avg_time": np.mean([r["time"] for r in results]),
 "p95_time": np.percentile([r["time"] for r in results], 95) if results else 0

 # Tool usage
 "tool_usage": self._analyze_tool_usage(results),

 # Error analysis
 "error_breakdown": self._analyze_errors(results)
 }

def _analyze_tool_usage(self, results: List[Dict]) -> Dict:
 """Analyze which tools are used and how often"""
 from collections import Counter

 all_tools = []
 for result in results:
 all_tools.extend(result["tools_used"])

 tool_counts = Counter(all_tools)

 return {
 "total_tool_calls": len(all_tools),
 "unique_tools": len(tool_counts),
 "most_common": tool_counts.most_common(5),
 "avg_tools_per_query": len(all_tools) / len(results) if results else 0
 }

def _analyze_errors(self, results: List[Dict]) -> Dict:
 """Analyze error patterns"""
 from collections import Counter

 errors = [r["error_type"] for r in results if r["error_type"]]
 error_counts = Counter(errors)

 return {
 "total_errors": len(errors),

```

```

 "error_rate": len(errors) / len(results) if results else 0,
 "error_types": dict(error_counts)
 }

 def generate_report(self, evaluation: Dict) -> str:
 """Generate human-readable evaluation report"""

 agg = evaluation["aggregated"]

 report = f"""
AGENT EVALUATION REPORT
{'-'*80}

Test Cases: {len(evaluation['individual_results'])}

PERFORMANCE METRICS
{'-'*80}
Success Rate: {agg['success_rate']:.1%}
Correctness Rate: {agg['correctness_rate']:.1%}

EFFICIENCY METRICS
{'-'*80}
Avg Iterations: {agg['avg_iterations']:.1f}
Avg Iterations (successful): {agg['avg_iterations_successful']:.1f}

COST METRICS
{'-'*80}
Avg Cost per Query: ${agg['avg_cost']:.3f}
Total Cost: ${agg['total_cost']:.2f}

LATENCY METRICS
{'-'*80}
Avg Time: {agg['avg_time']:.2f}s
P95 Time: {agg['p95_time']:.2f}s

TOOL USAGE
{'-'*80}
Total Tool Calls: {agg['tool_usage']['total_tool_calls']}
Avg Tools/Query: {agg['tool_usage']['avg_tools_per_query']:.1f}

Most Used Tools:
"""

 for tool, count in agg['tool_usage']['most_common']:
 report += f" {tool}: {count} times\n"

```

```

 if agg['error_breakdown']['error_types']:
 report += f"""\n
ERROR ANALYSIS
{'-'*80}
Total Errors: {agg['error_breakdown']['total_errors']} ({agg['error_breakdown']['error_r

Error Types:
"""

 for error_type, count in agg['error_breakdown']['error_types'].items():
 pct = count / len(evaluation['individual_results']) * 100
 report += f" {error_type}: {count} ({pct:.1f}%)\\n"

 return report

Usage
evaluator = AgentEvaluator()

Create test cases
test_cases = [
 {
 "query": "What is 15% of 200?",
 "expected_answer": "30"
 },
 {
 "query": "Search for the population of Tokyo and calculate the population densit
 "expected_answer": "6000" # approximate
 },
 # ... more test cases
]

Evaluate
evaluation = evaluator.evaluate_agent(agent, test_cases)

Generate report
report = evaluator.generate_report(evaluation)
print(report)

```

**Deliverable:** Agent evaluation with:

- Success rate metrics
- Efficiency metrics (iterations, cost, time)
- Tool usage analysis
- Error pattern detection

## Day 18-19: Production Monitoring

python

```
monitoring/agent_monitor.py
from typing import Dict, List
from collections import deque
import time

class AgentMonitor:
 """Monitor agent behavior in production"""

 def __init__(self, window_size: int = 100):
 self.window_size = window_size

 # Sliding windows
 self.recent_runs = deque(maxlen=window_size)
 self.recent_errors = deque(maxlen=window_size)

 # Counters
 self.total_runs = 0
 self.total_successes = 0
 self.total_cost = 0.0

 def log_run(self, result: Dict):
 """Log agent run"""
 self.total_runs += 1

 if result.get("success"):
 self.total_successes += 1
 else:
 self.recent_errors.append({
 "timestamp": time.time(),
 "query": result.get("query"),
 "error": result.get("error"),
 "scratchpad": result.get("scratchpad")
 })

 self.total_cost += result.get("cost", 0)

 # Add to sliding window
 self.recent_runs.append({
 "timestamp": time.time(),
 "success": result.get("success", False),
 "iterations": result.get("iterations", 0),
```

```

 "cost": result.get("cost", 0),
 "time": result.get("time", 0)
 })

 # Check for anomalies
 self.check_anomalies()

def check_anomalies(self):
 """Check for anomalous behavior"""

 if len(self.recent_runs) < 10:
 return # Not enough data

 recent = list(self.recent_runs)[-20:]

 # Check success rate
 recent_success_rate = sum(1 for r in recent if r["success"]) / len(recent)
 if recent_success_rate < 0.5:
 print(f"⚠️ ALERT: Low success rate: {recent_success_rate:.1%}")

 # Check cost
 recent_avg_cost = np.mean([r["cost"] for r in recent])
 overall_avg_cost = self.total_cost / self.total_runs

 if recent_avg_cost > overall_avg_cost * 1.5:
 print(f"⚠️ ALERT: Cost spike: ${recent_avg_cost:.3f} (baseline: ${overall_avg_cost:.3f})")

 # Check latency
 recent_p95_time = np.percentile([r["time"] for r in recent], 95)
 if recent_p95_time > 30:
 print(f"⚠️ ALERT: High latency: P95={recent_p95_time:.1f}s")

def get_dashboard_metrics(self) -> Dict:
 """Get metrics for dashboard"""

 if not self.recent_runs:
 return {}

 recent = list(self.recent_runs)

 return {
 "total_runs": self.total_runs,
 "success_rate": self.total_successes / self.total_runs if self.total_runs > 0 else 0,
 "recent_success_rate": sum(1 for r in recent if r["success"]) / len(recent),
 "avg_cost": self.total_cost / self.total_runs if self.total_runs > 0 else 0,
 "recent_avg_cost": np.mean([r["cost"] for r in recent]),
 }

```

```

 "avg_iterations": np.mean([r["iterations"] for r in recent]),
 "avg_time": np.mean([r["time"] for r in recent]),
 "p95_time": np.percentile([r["time"] for r in recent], 95) if recent else 0,
 "recent_errors": len(self.recent_errors),
 "error_rate": len(self.recent_errors) / len(recent) if recent else 0
 }

def get_recent_errors(self, n: int = 10) -> List[Dict]:
 """Get most recent errors"""
 return list(self.recent_errors)[-n:]

Usage with agent
monitor = AgentMonitor()

Wrap agent
class MonitoredAgent:
 def __init__(self, agent, monitor):
 self.agent = agent
 self.monitor = monitor

 def run(self, query: str) -> Dict:
 result = self.agent.run(query)
 self.monitor.log_run(result)
 return result

monitored_agent = MonitoredAgent(agent, monitor)

Use in production
for user_query in production_queries:
 result = monitored_agent.run(user_query)
 # ... return to user

Check dashboard
metrics = monitor.get_dashboard_metrics()
print(f"Success Rate: {metrics['success_rate']:.1%}")
print(f"Avg Cost: ${metrics['avg_cost']:.3f}")
print(f"Avg Time: {metrics['avg_time']:.2f}s")

Check errors
recent_errors = monitor.get_recent_errors()
if recent_errors:
 print(f"\nRecent Errors ({len(recent_errors)}):")
 for error in recent_errors:
 print(f" {error['error']}: {error['query'][:50]}")

```

**Deliverable:** Production monitoring with:

- Real-time metrics
- Anomaly detection
- Error tracking
- Dashboard metrics

**Day 20-21: Portfolio Documentation**

**Create comprehensive README:**

markdown

```
Production Agent System

Overview
ReAct-based agent framework for multi-step task completion with tool use, achieving 87%

Architecture
```

User Query

↓

ReAct Agent Loop:

- └ Reasoning (LLM decides next step)
- └ Action (Execute tool)
- └ Observation (Get result)
- └ Repeat until complete

↓

Final Answer (or Error + Partial Result)

```
Results

Performance Metrics (100 test cases)
- **Success Rate:** 87%
- **Correctness:** 92% (of successful runs)
- **Avg Iterations:** 5.2 (optimal: 4.1)
- **Avg Cost:** $0.042 per query
- **P95 Latency:** 8.3s

Tool Usage
- Most used: web_search (45%), calculator (30%), final_answer (25%)
- Avg tools per query: 3.1
- Tool error rate: 2.3%
```

### ### Failure Analysis

- Max iterations reached: 8%
- Infinite loop detected: 3%
- Tool execution errors: 2%

## ## Features

### ### 1. ReAct Pattern

Reasoning + Acting for systematic task completion:

Query: "What is 15% tip on \$85 meal?"

Thought: I need to calculate 15% of \$85

Action: calculator("85 \* 0.15")

Observation: 12.75

Thought: The tip is \$12.75

Action: final\_answer({"answer": "\$12.75"})

Result: \$12.75 (1 tool call, \$0.002 cost)

### ### 2. Controlled Execution

- Max iterations: 10
- Budget limit: \$0.50
- Timeout: 30s
- Loop detection

### ### 3. Tool Library

- `web\_search`: Current information
- `calculator`: Math calculations
- `read\_pdf`: Document extraction
- `send\_email`: Communications (supervised)

### ### 4. Error Handling

- Automatic retries (3x)
- Timeouts (30s per tool)
- Parameter validation
- Clear error messages

### ### 5. Production Monitoring

- Real-time success rate
- Cost tracking
- Latency metrics
- Anomaly detection



```

Usage

Basic Agent
```python
from agents import ReActAgent
from tools import search_tool, calculator_tool

agent = ReActAgent(
    tools=[search_tool, calculator_tool],
    max_iterations=10
)

result = agent.run("What is the population of Tokyo divided by the population of Paris?")
print(result["answer"])

```

Controlled Agent

```

from agents import ControlledReActAgent

agent = ControlledReActAgent(
    tools=[...],
    max_iterations=10,
    max_cost=0.50,
    timeout=30
)

result = agent.run("Complex query")

if "error" in result:
    print(f"Failed: {result['error']}")
    print(f"Partial: {result['partial_result']}")

```

Supervised Agent

```

from agents import SupervisedAgent

agent = SupervisedAgent(tools=[...])
agent.mark_tool_as_high_risk("send_email", ActionRisk.HIGH)
agent.set_approval_callback(human_approval_ui)

```

```
result = agent.run("Send email to client with proposal")
# Human approves email before sending
```

Design Decisions

Why ReAct over Chain-of-Thought?

- Chain-of-Thought: 45% success (can't access tools)
- ReAct: 87% success (reasoning + tools)
- Improvement: 93% relative gain

Why Function Calling over String Parsing?

- String parsing: 12% tool call failures (format errors)
- Function calling: 2% tool call failures (parameter errors)
- Improvement: 83% error reduction

Why Iteration Limits?

Without limits:

- 5% of queries loop infinitely
- Average cost \$2.50 (50x higher)

With limits (max_iterations=10):

- 3% hit limit (acceptable)
- Average cost \$0.042
- 98% cost reduction on failure cases

Tool Design Best Practices

Good Tool

```
{
  "name": "web_search",
  "description": ""Search the web for current information."
```

Use this when you need:

- Current events or news
- Real-time data

Do NOT use for:

- Mathematical calculations (use calculator)

Examples:

- "What's the weather in Paris?"
- "Who won the 2024 Super Bowl?"

```
""",
  "parameters": {
    "query": {
      "type": "string",
      "description": "Search query (2-10 words work best)"
    }
  }
}
```

✗ Bad Tool

```
{
  "name": "search",
  "description": "Search for something", # Too vague!
  "parameters": {
    "q": {"type": "string"} # No description!
  }
}
```

Evaluation Framework

Agent-Specific Metrics

- **Success Rate:** Can agent complete the task?
- **Correctness:** Is the answer correct?
- **Efficiency:** How many steps to solution?
- **Cost:** How much does it cost?

Test Cases

- Simple queries (1-2 steps): 40 cases
- Medium queries (3-5 steps): 35 cases
- Complex queries (6+ steps): 25 cases

Production Lessons

1. 80% of value is in tool design

- Clear descriptions reduce confusion by 60%
- Examples prevent 90% of misuse

1. Control mechanisms are essential

- Max iterations prevent runaway costs
- Timeouts prevent stuck agents
- Loop detection catches 95% of infinite loops

1. Monitoring catches issues early

- Success rate drop from 87% → 75% in 2 hours
- Root cause: Web search API down
- Resolution: 15 minutes (vs 2+ hours without monitoring)

1. Partial results save user frustration

- 13% of failures have useful partial results
- Users prefer "I found X but not Y" over "Error"

...

...

4. Blind Spots & Underrated Perspectives

Blind Spot 1: Agents Are Not Always the Answer

Common hype: "Use agents for everything!"

Reality: Agents add complexity and reduce reliability. Use only when necessary.

Decision framework:

python

```
# When NOT to use agents:
if can_answer_from_training_data(query):
    return single_turn_llm(query) # Don't need tools

if is_simple_rag_query(query):
    return rag_system(query) # RAG is simpler and more reliable

if requires_99_reliability(query):
    return deterministic_system(query) # Agents are ~85% reliable

# When TO use agents:
if requires_external_tools(query):
    return agent(query) # e.g., "What's the weather?"

if requires_multi_step_reasoning_and_tools(query):
    return agent(query) # e.g., "Book cheapest flight"
```

Blind Spot 2: Tool Descriptions Matter More Than You Think

Common teaching: "Just list the tools."

Reality: LLMs heavily rely on tool descriptions. Bad descriptions = wrong tool selection.

Example:

python

```
# Bad tool description
"search_database: Search the database"

# Result: Agent uses search_database for web searches (wrong!)

# Good tool description
"""search_database: Search the internal company database for records.

Use this for:
- Finding customer orders
- Looking up product inventory
- Retrieving employee information
```

```
Do NOT use for:
```

- Web searches (use web_search instead)
 - Current events (use web_search)
 - Mathematical calculations (use calculator)
- ```
"""
```

```
Result: Agent uses correct tool 95% of the time (vs 60% before)
```

## Blind Spot 3: Agents Need Different Evaluation Than LLMs

**Common mistake:** Using LLM evaluation metrics for agents.

**Reality:** Task completion matters more than answer quality.

python

```
LLM evaluation: Did it give a good answer?
llm_metrics = {
 'fluency': 0.95,
 'coherence': 0.92,
 'relevance': 0.89
}

Agent evaluation: Did it complete the task?
agent_metrics = {
 'task_success': 0.87, # Most important!
 'tool_correctness': 0.94,
 'efficiency': 5.2/4.1, # Actual vs optimal steps
 'cost': 0.042
}
```

## Blind Spot 4: Infinite Loops Are More Common Than You Think

**Reality:** Without proper controls, 5-10% of agent runs loop infinitely.

**Common causes:**

- Tool returns unhelpful result, LLM tries again
- LLM doesn't know when to give up
- Goal is impossible but LLM doesn't realize

**Solution:** Loop detection + forced intervention (as shown in code examples above)

## Blind Spot 5: Users Need to See Agent Progress

**Common mistake:** Only showing final answer.

**Reality:** Multi-step tasks take time. Users need to see progress.

python

```
Bad: Silent execution
result = agent.run(query) # User waits... no feedback
print(result["answer"])

Good: Show progress
def agent_with_streaming(query):
 yield "🤔 Thinking..."

 for step in agent.run_with_streaming(query):
 if step.type == "thought":
 yield f"💭 {step.content}"
 elif step.type == "tool_call":
 yield f"🔧 Using {step.tool}..."
 elif step.type == "observation":
 yield f"📊 Result: {step.content[:100]}"

 yield f"✅ {final_answer}"
```